

# Vector Indexing and Slicing

You can use bracket notation to index and access individual elements from a vector:

In [11]:

```
v1 <- c(100, 200, 300)
v2 <- c('a', 'b', 'c')
```

In [12]:

```
v1
v2
```

Out[12]:

```
100 200 300
```

Out[12]:

```
'a' 'b' 'c'
```

Indexing works by using brackets and passing the index position of the element as a number. Keep in mind index starts at 1 (in some other programming languages indexing starts at 0).

In [13]:

```
# Grab second element
v1[2]
```

Out[13]:

```
200
```

In [14]:

```
v2[2]
```

Out[14]:

```
'b'
```

## Multiple Indexing

We can grab multiple items from a vector by passing a vector of index positions inside the square brackets. For example:

In [17]:

```
v1[c(1, 2)]
```

Out[17]:

```
100 200
```

In [19]:

```
v2[c(2, 3)]
```

```
Out[19]:
```

```
'b' 'c'
```

```
In [20]:
```

```
v2[c(1,3)]
```

```
Out[20]:
```

```
'a' 'c'
```

## Slicing

You can use a colon (:) to indicate a slice of a vector. The format is:

```
vector[start_index:stop_index]
```

and you will get that "slice" of the vector returned to you. For example:

```
In [21]:
```

```
v <- c(1,2,3,4,5,6,7,8,9,10)
```

```
In [22]:
```

```
v[2:4]
```

```
Out[22]:
```

```
2 3 4
```

```
In [23]:
```

```
v[7:10]
```

```
Out[23]:
```

```
7 8 9 10
```

Notice how the element at both the starting index and the stopping index are included.

## Indexing with Names

We've previously seen how we can assign names to the elements in a vector, for example:

```
In [26]:
```

```
v <- c(1,2,3,4)
names(v) <- c('a', 'b', 'c', 'd')
```

We can use those names along with the indexing brackets to grab individual elements from the array!

```
In [27]:
```

```
v['a']
```

```
Out[27]:
```

Out[27]:

**a:** 1

Or pass in a vector of names we want to grab:

In [28]:

```
# Notice how we can call out of order!  
v[c('a','c','b')]
```

Out[28]:

**a**  
1  
**c**  
3  
**b**  
2

## Comparison Operators and Selection

As alluded to in the comparison operator lecture, we can use comparison operators to filter out elements from a vector. Sometimes this is referred to as boolean/logical masking, because you are creating a vector of logicals to filter out results you want. Let's see an example of this:

In [29]:

```
v
```

Out[29]:

**a**  
1  
**b**  
2  
**c**  
3  
**d**  
4

In [30]:

```
v[v>2]
```

Out[30]:

**c**  
3  
**d**  
4

Let's break this down to see how it works, we first get the vector v>2:

In [31]:

```
v>2
```

Out[31]:

Out[31]:

```
a  
FALSE  
b  
FALSE  
c  
TRUE  
d  
TRUE
```

Now we basically pass this vector of logicals through the brackets of the vector and only return true values at the matching index positions:

In [32]:

```
v[v>2]
```

Out[32]:

```
c  
3  
d  
4
```

We could also assign names to these logical vectors and pass them as well, for example:

In [33]:

```
filter <- v>2
```

In [34]:

```
filter
```

Out[34]:

```
a  
FALSE  
b  
FALSE  
c  
TRUE  
d  
TRUE
```

In [35]:

```
v[filter]
```

Out[35]:

```
c  
3  
d  
4
```

Okay that is it for the basics of vectors! Up next is an exercise for review!